Qwen

From Classic Code to Canonical Patterns: A Rust-Based Framework for Replicating GoldenEye's Stealth Math

This research report details a comprehensive strategy for reverse-engineering, formalizing, and implementing late-1990s console game design patterns, with a primary focus on the stealth and mission systems of Rare's GoldenEye 007 and Perfect Dark [2] [3]. The core objective is to translate documented techniques, such as the STAN/PAD room-based visibility system, vision cones, sound propagation radii, and awareness accumulation mechanics, into precise mathematical models. These models are then realized as deterministic, testable Rust Entity Component System (ECS) modules. Each module is governed by strict, machine-readable JSON or TOML schemas that control its parameters, ensuring predictability and facilitating reliable unit testing. Finally, each implemented pattern is registered within a knowledge graph, providing an AI-powered development assistant with the context needed to discover, understand, and reuse these canonical design patterns effectively. This document provides a deep analysis of the reconstruction process, focusing on the immediate priority of creating a complete, reference-quality stealth AI module before expanding the methodology to other classic mechanics.

Formalizing Perception: From STAN/PAD Room-Based Visibility to Modern Navigation Graphs

The perception systems of classic first-person shooters like GoldenEye 007 were defined by significant hardware limitations, which necessitated clever architectural optimizations. The most prominent of these was the use of STANs (Spatial Partitioning Trees) and PADs (Visibility Data) to manage how non-player characters (NPCs) perceived the player [2] [3]. A STAN represented a distinct room or area in a level, while a PAD was a pre-calculated matrix that determined which STANs could see each other. This allowed the game engine to perform a simple, fast lookup to determine if a guard in one room could have a line of sight to the player in another, completely avoiding the computationally expensive task of

performing real-time 3D raycasting between all objects in the scene [4]. This technique is a prime example of a coarse-grained, data-driven optimization that prioritizes performance by simplifying spatial reasoning at the cost of perfect geometric accuracy. The goal of this research is to reconstruct this logic not as a direct emulation of the original code, but as a modern, deterministic mathematical model that captures the underlying intent and behavior.

To implement this concept within a contemporary Rust ECS framework, the STAN/PAD system can be reinterpreted as a "Bond-style nav and perception" system. Instead of replicating the original binary format, the level designer would define a coarse navigation graph where nodes represent rooms or zones, and edges represent adjacency or clear lines of sight. This graph becomes a crucial data asset for the AI. The runtime logic would involve two main checks: a fine-grained check for direct line-of-sight within a single zone (the equivalent of looking through a window in the same room), and a coarse-grained check against the pre-computed adjacency graph to determine if a target zone is even visible from the current one. The conversation history provides a strong foundation for the fine-grained check through the `within_vision_cone` function, which determines if a player is within a guard's field of view based on direction and an angle threshold .

The next logical step is to integrate the coarse-grained check. This involves extending the `DetectionInputs` struct to include information about the source and target STAN IDs . The `tick_guard_stealth` function would then be modified to first query the loaded adjacency graph. If the graph indicates that the target STAN is visible, the function proceeds to the fine-grained checks (vision cone and line-of-sight). If not, the instantaneous visibility score `V` is immediately set to zero, as the guard cannot perceive the player, regardless of any other factors. This modular approach cleanly separates the high-level world partitioning from the low-level perception calculations, making the entire system highly flexible and data-driven. The adjacency data itself can be stored in a structured format, such as a JSON file defining the graph's connectivity, which is loaded at level initialization. This aligns perfectly with the overarching project goal of using explicit schemas to define game mechanics.

The following table outlines the conceptual transformation from the original GoldenEye technique to its modern implementation blueprint:

Feature	Original GoldenEye Implementation	Proposed Modern Implementation
Spatial Partitioning	Level geometry was divided into rooms and areas, referred to as STANs.	A coarse navigation graph is defined, where nodes represent zones or rooms and edges represent adjacency or clear sightlines.
Visibility Logic	A Pre-Calculated Visibility Matrix (PAD) determined which STANs had a direct line of sight to each other.	A data-driven adjacency graph is loaded at level start. Line-of-sight is checked via a simple lookup in this graph before detailed checks.
Runtime Check	The engine performed a fast matrix lookup to see if the player's STAN was visible to the guard's STAN.	An ECS system queries the loaded graph. If a link exists, it triggers more detailed, localized checks (e.g., vision cone, object occlusion).
Performance Optimization	Avoided expensive per-frame, per-object 3D raycasting across the entire level.	Reduces the number of detailed visibility checks by only considering targets in adjacent, known-visible zones.
Tunability	Hardcoded in the level data by level designers; not exposed to runtime modification.	The adjacency graph and its associated properties (e.g., 'is_doorway', 'has_window') can be defined in a JSON schema, allowing for dynamic changes or modding.

This formalization directly addresses the user's request to "reconstruct specific N64/PS1 technical patterns" and express them as "deterministic mathematical models." The STAN/PAD concept is not just replicated; it is abstracted into a more generalizable pattern—a visibility graph—that can be applied to various genres and platforms beyond just N64-style games. By anchoring this model in a well-defined data contract (the JSON schema for the navigation graph), the system ensures that any changes to the level's perceptual topology are explicit, validated, and predictable. This prevents the kind of accidental bugs that can arise from manually editing complex binary data, a common challenge in traditional ROM hacking workflows [33]. The integration of this graph into the broader stealth AI module will be crucial for enabling behaviors like guards investigating sounds or sights originating from another room, adding a layer of strategic depth that goes beyond simple, static patrols. The final implementation would require a loader component in the ECS that processes the navigation graph data during the world initialization phase, populating a resource that the AI systems can efficiently query every frame .

The Mathematical Core of Stealth: Deterministic Awareness and State Transitions

At the heart of the GoldenEye and Perfect Dark stealth experience lies a sophisticated yet elegant mathematical model for enemy awareness. This model moves beyond simple binary states (e.g., "guard sees player" vs. "guard does not see player") and instead introduces a continuous, accumulating metric that drives a sequence of discrete

behavioral states. This approach creates a more believable and dynamic AI, where a guard's reaction to a potential threat evolves over time. The provided materials offer a clear and concrete formulation of this model, which serves as the foundational mathematics for the proposed Rust ECS module. The core of this system is a differential equation that governs the change in a guard's awareness meter over time, combined with fixed thresholds that trigger state changes.

The instantaneous visibility of the player to a guard, denoted as V, is the input to this awareness system. It is a composite score derived from multiple factors, including distance, light levels, the player's posture, and movement state . The formula for visibility is expressed as a product of these individual modifiers:

$$V = B_{\text{base}} \cdot f_d(d) \cdot f_L(L) \cdot f_P(P) \cdot f_M(M)$$

Here, B_{base} is a constant representing the base detectability, $f_d(d)$ is a distance falloff function, $f_L(L)$ is a function of ambient light, $f_P(P)$ accounts for the player's posture (standing, crouched, prone), and $f_M(M)$ accounts for movement state (still, walking, running) . This multiplicative model allows for intuitive tuning; for instance, being spotted while running in bright light results in a much higher visibility score than being prone in darkness.

With the visibility score V calculated, the awareness meter A is updated on each game tick. The provided materials specify this update rule as:

$$A(t+\Delta t) = \max\left(0, A(t) + V_{\text{total}}\Delta t - k_{\text{decay}}\Delta t\right)$$

where $A(t)$ is the awareness at the previous timestep, Δt is the time delta, V_{total} is the total visibility score (which will later include a sound component, V_{sound}), and k_{decay} is the awareness decay constant . This formula has several key implications. First, awareness increases when the player is visible (V > 0) and decreases over time when they are not ($V=0$). Second, the rate of increase and decrease is explicitly controlled by the parameters V_{total} and k_{decay}. Third, the result is clamped between 0 and 1, ensuring the awareness meter remains a normalized value.

The continuous awareness value A is then used to determine the guard's discrete behavioral state. This is achieved through a series of hard-coded thresholds, which act as the invariants of the system. The conversation outlines three primary states: Idle, Suspicious, and Alert . The state transitions are governed by the following conditions:

- If $A < T_{\text{suspicious}}$, the guard is in the `Idle` or `Patrol` state.

- If $T_{\text{suspicious}} \leq A < T_{\text{alert}}$, the guard enters the `Suspicious` or `Investigate` state, beginning to look around or move towards the last seen location.
- If $A \geq T_{\text{alert}}$, the guard enters the `Alert` or `Attack` state, actively searching for and pursuing the player.

These thresholds, $T_{\text{suspicious}}$ and T_{alert}, are fundamental parameters that define the aggressiveness and paranoia of the AI. By exposing these values in a data schema, level designers or AI agents can tune the overall feel of the stealth gameplay without altering any of the underlying C++ or Rust code . For example, a lower T_{alert} would create a more challenging, "spooky" experience, while a higher value would allow for more reckless player behavior.

The mathematical model also extends to other sensory inputs, most notably sound. While the initial implementation focuses on vision, the long-term plan includes integrating a separate sound-based visibility score, V_{sound} . This score would be generated based on the loudness of player actions (e.g., footsteps, weapon fire) and would propagate through the environment based on a different set of rules than visual detection. The total visibility is then simply the sum of the vision and sound scores:

$$V_{\text{total}} = V_{\text{vision}} + V_{\text{sound}}$$

This additive model allows for different types of threats to contribute independently to the guard's awareness. A loud gunshot might cause a large spike in V_{sound}, quickly pushing the guard to an alert state, while a visually-camouflaged player moving slowly might only generate a small amount of V_{vision}. The elegance of this mathematical framework lies in its simplicity and composability. It breaks down a complex emergent behavior (a guard reacting intelligently to stealth) into a set of smaller, independent, and tunable mathematical functions. This makes the system both easy to reason about and straightforward to implement in a deterministic fashion, which is essential for the project's goals. The subsequent section will detail how this mathematical core is translated into a concrete Rust ECS module with strict data contracts.

Implementing the Stealth AI Module: A Rust ECS Blueprint with Strict Data Contracts

Translating the mathematical models of the GoldenEye/Perfect Dark stealth system into a robust and maintainable software architecture requires careful design choices within the

Rust programming language and its ecosystem. The recommended approach is to encapsulate the entire stealth AI logic within a dedicated Rust module, tentatively named `bondfpscore::stealth_ai`, adhering to the principles of a deterministic ECS . This module will serve as a self-contained system that manages the state of enemies and updates it based on player actions, all governed by a strict set of data contracts defined in JSON schemas. This section provides a detailed blueprint for this implementation, outlining the necessary structs, enums, and functions, and emphasizing the importance of unit tests to enforce the mathematical invariants discussed previously.

The core of the module will revolve around defining ECS components and resources. The primary component tracking the AI's internal state will be `GuardStealthState`. This struct will hold the continuous awareness value and the discrete state of the guard.

```
/// Discrete guard awareness state, driven by the continuous awareness met
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum AwarenessState {
    Idle,
    Suspicious,
    Alert,
}

/// Per-guard stealth state — store this as a component.
#[derive(Debug, Clone, Copy, Default)]
pub struct GuardStealthState {
    /// Normalized [0, 1] awareness meter.
    pub awareness: f32,
    pub state: AwarenessState,
}
```

This component is minimal and focused, containing only the state variables that need to be persisted between frames. The actual logic for updating this state will reside in a pure Rust function, `tick_guard_stealth`, which takes the current state and inputs, and returns the new state. This functional approach reinforces determinism, as the output is solely dependent on the inputs, with no hidden state or side effects.

The inputs to this function are gathered from various sources in the ECS. They are bundled into a `DetectionInputs` struct.

```
/// Continuous detection inputs for a single guard evaluating a single pla
#[derive(Debug, Clone, Copy)]
```

```
pub struct DetectionInputs {
    /// Guard eye position.
    pub guard_pos: Vec3,
    /// Guard forward direction (normalized).
    pub guard_forward: Vec3,
    /// Player position (at head/torso).
    pub player_pos: Vec3,
    /// Ambient light at player [0, 1].
    pub light_level: f32,
    pub posture: Posture,
    pub movement: MovementState,
    /// Whether there is a clear line of sight.
    pub has_line_of_sight: bool,
}
```

This struct acts as a data bundle, collecting all the necessary information for the perception calculation in one place, which simplifies the function signature and improves readability.

The final piece of the puzzle is the `StealthParams` struct. This is arguably the most important part of the data contract, as it contains all the tunable parameters that define the stealth behavior. These parameters should not be hardcoded but loaded from a data file at runtime.

```
/// Tunable stealth parameters. These should be loaded from data (JSON/Lua
/// and cached in a resource in your ECS world.
#[derive(Debug, Clone)]
pub struct StealthParams {
    pub base_visibility: f32,
    pub max_distance: f32,
    pub distance_exponent: f32,
    pub light_exponent: f32,
    pub posture_standing: f32,
    pub posture_crouched: f32,
    pub posture_prone: f32,
    pub move_still: f32,
    pub move_walking: f32,
    pub move_running: f32,
    pub awareness_decay: f32,
    pub threshold_suspicious: f32,
```

```
    pub threshold_alert: f32,
}

impl Default for StealthParams {
    fn default() -> Self {
        // ... default values as specified in the conversation ...
    }
}
```

This struct will be populated by a loader function that parses a JSON file, validating its contents against a predefined schema before construction. This ensures that invalid or out-of-bounds parameters cannot be loaded, preventing runtime errors and undefined behavior.

The core logic is contained within the `tick_guard_stealth` function, which orchestrates the entire process.

```
pub fn tick_guard_stealth(
    params: &StealthParams,
    guard_state: GuardStealthState,
    inputs: &DetectionInputs,
    dt: f32,
) -> GuardStealthState {
    let delta = inputs.player_pos.sub(inputs.guard_pos);
    let in_cone = within_vision_cone(inputs.guard_forward, delta, cone_ang

    let visibility = if in_cone && inputs.has_line_of_sight {
        compute_visibility(params, inputs)
    } else {
        0.0
    };

    update_awareness(params, guard_state, visibility, dt)
}
```

This function demonstrates a clean separation of concerns. It first performs the high-level checks (vision cone and line-of-sight), then computes the visibility score if those checks pass, and finally updates the awareness state based on that score. The helper functions (`compute_visibility`, `update_awareness`) contain the specific mathematical implementations of the formulas discussed earlier .

Finally, this entire module must be underpinned by a rigorous suite of unit tests to verify its deterministic behavior and enforce the mathematical invariants. These tests should cover scenarios such as: 1. **No Stimulus:** Verify that given an initial awareness of 0 and no visibility, the awareness remains 0 over many timesteps. 2. **Constant Stimulus:** Verify that with constant visibility, the awareness meter increases linearly until it reaches the maximum value of 1. 3. **Threshold Crossing:** Verify that the awareness meter correctly crosses the `threshold_suspicious` and `threshold_alert` boundaries, causing the `state` to change accordingly. 4. **Decay Test:** Verify that after visibility drops to zero, the awareness meter decays back to zero at the correct rate determined by `k_decay`. 5. **Parameter Sensitivity:** Test that changing parameters like `base_visibility` or `awareness_decay` in the `StealthParams` struct leads to proportionally different behavior in the resulting awareness trajectory.

By implementing the module this way—with clear components, a pure logic function, a centralized data resource, and a comprehensive test suite—the project achieves its primary goals. The implementation is small, focused, and testable. The data contracts are explicit and strictly enforced. And the entire system is grounded in the precise mathematical models derived from the classic games it seeks to emulate.

Data-Driven Design: Schemas and Knowledge Graphs for AI-Chat Discoverability

Creating a functional and mathematically sound stealth AI module is only half the battle. To fulfill the ultimate research goal of building a system that can be reliably manipulated by an AI agent, the module must be made discoverable, understandable, and accessible. This is achieved through a dual strategy of formal data schemas and deep integration with a knowledge graph. The schemas provide a machine-readable contract for the module's parameters, while the knowledge graph provides a semantic map of the entire project, linking concepts to their concrete implementations. This combination transforms the module from a mere collection of Rust files into a first-class citizen in the AI's development environment.

The first step is to define a strict JSON Schema for the `StealthParams` resource. This schema, which can be stored as `docs/schema/bond_stealth.schema.json`, serves as the definitive specification for how to configure the stealth system . It documents the purpose of each parameter and enforces constraints like minimum/maximum values and ordering relationships. The provided materials outline a detailed structure for this

schema, which includes global parameters, posture-specific factors, movement factors, and settings for a future sound system . This schema is critical because it prevents the AI from generating nonsensical or invalid configurations. For example, it can enforce that `t_alert` is always greater than `t_suspicious`, a crucial invariant for the intended state machine behavior. The existence of this schema also enables automated validation in a CI pipeline, where any pull request modifying a data file for this module can be automatically checked against the schema to prevent regressions .

The second, and perhaps more transformative, element is the integration with the knowledge graph. The knowledge graph acts as the central nervous system for the AI, providing it with context about the available tools, modules, and assets in the project. For the newly created `bondfpscore::stealth_ai` module, a corresponding `SystemNode` must be added to the graph, typically in a file like `knowledgegraph/systems.json` . This node acts as a rich metadata descriptor for the module.

The following table details the fields of this `SystemNode` and their significance:

Field	Example Value	Significance
`id`	`systems.bondfpscore.stealth_ai`	A globally unique identifier for the AI to reference this system. Its dot-separated structure implies a hierarchical relationship (e.g., under `bondfpscore`).
`kind`	`System`	Specifies the type of node in the graph, distinguishing it from assets or recipes.
`tags`	`["Deterministic", "LuaFacing", "Stealth", "BondLike"]`	A list of keywords that allow the AI to search for and filter systems based on semantic meaning.
`paths`	`["crates/bondfpscore/src/stealth_ai.rs", "data/bondfpscore/stealth/default.json", "docs/schema/bond_stealth.schema.json"]`	An array of repository-relative file paths that implement or relate to this system. This gives the AI the exact location of the source code, data, and schema.
`repo`	`Doctor0Evil/Nintendoor64`	The canonical GitHub repository hosting the files.
`description`	"GuardStealthState component, detection formulas, and tuning schema for Bond/PD-style stealth FPS."	A human-readable summary that provides context for the AI.
`provides`	`["Nintendoor64.Artifacts.Schema.StealthParams"]`	A list of capabilities or features this node exposes, allowing the AI to understand what it can ask for.
`depends_on`	`[]`	A list of other `SystemNode` IDs that this system relies on. This will grow as the stealth AI interacts with other systems like movement or audio.

By populating this node, the AI gains a powerful new capability. Instead of vaguely prompting, "make the guards smarter," it can now issue a targeted command like, "Update the parameters in `systems.bondfpscore.stealth_ai` to make the guards more paranoid." The AI can use the `paths` field to locate the relevant JSON file (`data/bondfpscore/stealth/default.json`) and the `description` and `tags` to confirm it has found the correct system. Furthermore, by referencing the `id`, it can programmatically query the knowledge graph for related systems, such as `systems.bondfpscore.missions`, to understand how the guard's state might affect the overall mission outcome. This turns the AI from a code generator into a true development assistant that understands the architecture and dependencies of the project it is working on.

This data-driven approach extends to the entire toolchain. The `starzip-cli` and `sonia-core` tools, for instance, are themselves registered as `SystemNode`s in the knowledge graph . This creates a fully interconnected web of knowledge where the AI can orchestrate the entire build process. A prompt like, "Modify the title screen of the game" could trigger a chain of actions: the AI finds the `starzip-cli` system to understand how to patch the ROM, locates the `bondfpscore` stealth module to ensure the change doesn't conflict with existing logic, and uses the `sonia-core` system to upload the new asset to the repository, initiating the build pipeline . This level of integration is the key to achieving the "next-level" development workflow envisioned in the research, mimicking the rapid iteration cycles of the N64/PS1 era but powered by modern, deterministic, and AI-assisted tooling . The formalization of these data contracts and their registration in the knowledge graph is what elevates the project from a simple codebase to a truly intelligent development platform.

Expanding the Pattern Library: Extending the Framework to Other Classic Mechanics

The successful reconstruction of the GoldenEye/Perfect Dark stealth AI module provides a powerful template for formalizing other classic game mechanics. The core principle—capturing a design pattern as a deterministic mathematical model, implementing it in a small, testable Rust ECS module, defining its parameters with a strict schema, and registering it in the knowledge graph—is a scalable methodology. The user's research goal explicitly mentions expanding this effort to other mechanics like lock-on targeting, damage/armor curves, and objective DAGs . This section explores how this template can

be applied to these specific examples, demonstrating the broader applicability of the framework.

Lock-On Targeting: The lock-on targeting system popularized by games like *GoldenEye* and later perfected in titles like *Perfect Dark* offers a compelling case study. The core mechanic involves selecting a target from a group of potential candidates based on angular alignment relative to the camera. This can be modeled mathematically using vector dot products. The algorithm for finding a target candidate `P_i` would be to select the entity whose forward vector `v_i` satisfies the condition that the dot product with the aim vector `a` is above a certain threshold: $\hat{p_i} \cdot \hat{a} \geq \cos(\theta_{\text{max}})$, where θ_{max} is the maximum cone angle for locking on . Among all candidates that satisfy this condition, the closest one is typically selected. This entire logic can be encapsulated in a Rust module, say `bondfpscore::lockon`. The module would expose a function `find_target(camera_forward: Vec3, candidates: &[Entity]) -> Option<EntityId>`. The parameters—most notably `theta_max`—would be defined in a corresponding JSON schema. This module would be registered in the knowledge graph as `systems.bondfpscore.lockon`, making it a discoverable component for any FPS or TPS module that requires targeting.

Damage and Armor Curves: The combat systems in these classic games were often surprisingly deep, featuring interlocking resistances and damage falloff. This can be formalized with a deterministic damage calculation formula. A common pattern is a base damage value modified by a resistance factor. The effective damage can be calculated as:

$$D_{\text{effective}} = D_{\text{base}} \cdot (1 - R_e)$$

where R_e is the effective resistance of the target, which could be a simple flat value or a combination of elemental resistances . For example, a projectile might have an elemental property (e.g., explosive, incendiary), and an enemy might have resistances to those elements. The `R_e` value would be a weighted average of these resistances. This formula is inherently deterministic and can be easily implemented in Rust. A `bondfpscore::combat` module could house the `calculate_damage` function. The parameters defining weapon damage values and enemy resistances would be managed through a set of JSON/TOML files. For instance, a `weapons.json` file could define the `D_base` for each weapon type, while an `enemies.json` file could define their resistance profiles. Each of these data files would have its own schema to ensure validity. This modular approach allows for complex interactions to be built from simple, composable mathematical rules.

Objective Directed Acyclic Graphs (DAGs): The mission structures in *Perfect Dark*, and to some extent *GoldenEye*, featured objectives that could often be completed in multiple orders, offering players meaningful choices. This structure is mathematically represented by a Directed Acyclic Graph (DAG). In this model, nodes represent individual objectives, and directed edges represent dependencies (i.e., "to complete objective B, you must first complete objective A"). A more advanced version adds conditions to the nodes, where an objective O_i is considered complete only if its condition $C_i(s)$ is true in the current game state `s` AND all of its parent objectives have been completed . The engine would need a system to evaluate the entire DAG. This system would iterate through the graph, checking the completion status of each node. A topological sort of the DAG can help organize this evaluation process. The definition of the mission—its nodes, edges, and conditions—would be stored in a JSON file, for example, `missions/m01.json`. The conditions `C_i(s)` could be implemented as simple Lua scripts or a constrained Domain-Specific Language (DSL) that the engine can safely evaluate. The knowledge graph would contain a node for the mission system, `systems.bondfpscore.mission_scripting`, pointing to the DAG evaluator code and the convention for mission definition files. This would allow the AI to generate or modify mission flows by manipulating the JSON structure, with the engine responsible for enforcing the DAG constraints.

By applying this consistent pattern—mathematical model, Rust module, schema, knowledge graph entry—to each classic mechanic, the project builds a library of "canonical" patterns. This library becomes a powerful asset, allowing developers (and AI agents) to construct complex, engaging gameplay by combining these proven, deterministic components rather than inventing new systems from scratch each time. It codifies decades of game design wisdom into a format that is both human-readable and machine-executable, forming the bedrock of a modern, AI-assisted retro game development platform.

Synthesis and Next Steps: Integrating Components into a Coherent Development Workflow

This research has successfully outlined a comprehensive framework for reconstructing and formalizing classic N64/PS1 game design patterns into a modern, deterministic, and AI-assisted development workflow. The central thesis is that by translating iconic mechanics like the GoldenEye/Perfect Dark stealth system into precise mathematical models, implementing them as small, testable Rust ECS modules, and enforcing their use through strict data schemas and knowledge graph integration, it is possible to create a

powerful and reliable system for game creation. The journey begins with a deep understanding of the source material, progresses through a rigorous engineering process of modeling and implementation, and culminates in a highly structured and discoverable asset library.

The analysis confirms that the GoldenEye STAN/PAD visibility system can be effectively modeled as a modern navigation and perception graph, separating high-level spatial reasoning from low-level line-of-sight checks. The core of the stealth AI is elegantly captured by a differential equation for awareness accumulation, driven by a composite visibility score and governed by discrete state thresholds. This mathematical core is eminently suitable for implementation in a Rust ECS module, `bondfpscore::stealth_ai`, which would feature clearly defined components (`GuardStealthState`), a pure logic function (`tick_guard_stealth`), and a central resource for tunable parameters (`StealthParams`). The enforcement of this design through a JSON schema for `StealthParams` and a corresponding `SystemNode` in the knowledge graph is not merely an administrative step; it is the mechanism that makes the entire system controllable and understandable by an AI agent. This process, of capturing a design pattern in a repeatable template, can be extended to other mechanics such as lock-on targeting, damage calculations, and mission scripting via objective DAGs, thereby building a library of canonical, reusable patterns.

To transition from this conceptual framework to a tangible, operational system, the following sequence of concrete actions is recommended:

1. **Implement the Core Stealth Module:** Begin by writing the complete Rust module `bondfpscore::stealth_ai` as outlined. This includes the `GuardStealthState`, `AwarenessState`, `DetectionInputs`, and `StealthParams` structs, along with the pure functions `tick_guard_stealth`, `compute_visibility`, and `update_awareness`. Ensure the implementation matches the mathematical formulas precisely.

2. **Develop the Data Contract and Loader:** Create the `docs/schema/bond_stealth.schema.json` file to formally specify the valid structure and constraints for the `StealthParams` resource . Subsequently, write a Rust loader function within the `bondfpscore` crate that reads a JSON file (e.g., `data/bondfpscore/stealth/default.json`), validates it against the schema using a library like `serde_json`, and populates the `StealthParams` resource in the ECS world. This loader should emit warnings for any detected inconsistencies, such as incorrect parameter ranges.

3. **Create a Rigorous Unit Test Suite:** Develop a comprehensive set of unit tests for the stealth module. These tests must empirically validate the mathematical invariants. Specifically, test that the awareness meter decays to zero in the absence of stimuli, rises linearly under constant visibility, and correctly transitions between `Idle`, `Suspicious`, and `Alert` states upon crossing the `threshold_suspicious` and `threshold_alert` boundaries.

4. **Integrate with a Minimal Testbed:** Set up a simple test level with a player character and one or more AI-controlled guards. Use the implemented stealth module to drive the guards' behavior. Manually test the system by observing the guards' state transitions as the player runs, walks, crouches, and enters/exits the guards' vision cones and sound radii. This practical validation is crucial for verifying that the theoretical model works as intended in a simulated environment.

5. **Build the AI-Facing Infrastructure:** With the module stable, create the knowledge graph entries. Add the `systems.bondfpscore.stealth_ai` `SystemNode` to `knowledgegraph/systems.json`, populating it with links to the source code, data files, and schema. Also, create a `systems.bondfpscore.missions` node to represent the planned objective DAG system. This makes the new pattern discoverable and usable by the AI development assistant.

6. **Expand Functionality Incrementally:** After the vision-based stealth is fully validated, expand the system to include sound propagation. Start with a basic model where different player actions (walking, running, weapon fire) produce different `V_sound` values that decay with distance. Integrate this into the `tick_guard_stealth` function to create a combined `V_total`.

By executing this plan, the project will move from a high-level research proposal to a functional and validated component of the larger game development platform. This completed module will serve as the definitive reference implementation for all future work, embodying the core principles of deterministic design, data-driven control, and AI-assisted development.

Reference

1. [PDF] Game Programming Gems 7 https://gaoyichao.com/Xiaotu//papers/Game%20Programming%20Gems%207.pdf
2. Steam 创意工坊::fill! download!!! https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3058480340
3. Steam 创意工坊::Half life alyx model replacements https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=2757813551
4. The Level Design Book - pdfcoffee.com https://pdfcoffee.com/the-level-design-book-pdf-free.html
5. Homebrew Core Formulae Overview | PDF | Command Line Interface https://www.scribd.com/document/551715438/Brew-Formula
6. Compare Packages Between Distributions - DistroWatch.com https://distrowatch.com/dwres.php?resource=compare-packages&firstlist=gentoo&secondlist=opensuse&firstversions=0&secondversions=0&showall=yes
7. Compare Packages Between Distributions - DistroWatch.com https://distrowatch.com/dwres.php?resource=compare-packages&firstlist=netbsd&secondlist=batocera&firstversions=0&secondversions=0&showall=yes
8. [PDF] Reinforcement-Learning-based Intrusion Detection ... - ResearchGate https://www.researchgate.net/publication/385166031_Reinforcement-Learning-Based_Intrusion_Detection_in_Communication_Networks_A_Review/fulltext/6718c68155a5271cded36bce/Reinforcement-Learning-Based-Intrusion-Detection-in-Communication-Networks-A-Review.pdf
9. An adaptive method and a new dataset, UKM-IDS20, for the network ... https://www.researchgate.net/publication/354505754_An_adaptive_method_and_a_new_dataset_UKM-IDS20_for_the_network_intrusion_detection_system
10. (PDF) Climate Change and Biodiversity in Maine: Vulnerability of ... https://www.researchgate.net/publication/272023400_Climate_Change_and_Biodiversity_in_Maine_Vulnerability_of_Habitats_and_Priority_Species
11. [PDF] Eva Pongrácz Victor Pavlov Niko Hänninen Editors - ResearchGate https://www.researchgate.net/profile/Eda-Ayaydin/publication/

339760549_Finnish_Sami_Is_Tourism_a_Preservation_of_Indigenous_Culture/links/63b6e124a03100368a565378/Finnish-Sami-Is-Tourism-a-Preservation-of-Indigenous-Culture.pdf

12. Arxiv今日论文| 2026-01-22 - 闲记算法 http://lonepatient.top/2026/01/22/arxiv_papers_2026-01-22
13. A Survey of Machine Learning for Computer Architecture and Systems https://dl.acm.org/doi/10.1145/3494523?sid=SCITRUS
14. bing.txt - FTP Directory Listing ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/bing.txt
15. (PDF) An Object-Oriented Parametric Framework for Generating Six ... https://www.researchgate.net/publication/390137071_An_Object-Oriented_Parametric_Framework_for_Generating_Six-Dimensional_Auditory_Environments_in_AI-Simulated_Architectural_Acoustics_for_Cinematic_Applications
16. Encyclopedia of Computer Graphics | PDF | Augmented Reality https://www.scribd.com/document/859605923/Encyclopedia-of-Computer-Graphics-and-Games-Springer-2024
17. Arxiv今日论文| 2026-03-05 - 闲记算法 http://lonepatient.top/2026/03/05/arxiv_papers_2026-03-05.html
18. Artificial Intelligence - arXiv https://arxiv.org/list/cs.AI/new
19. The 2025 Conference on Empirical Methods in Natural Language ... https://aclanthology.org/events/emnlp-2025/
20. The 2026 active metamaterials roadmap - IOPscience https://iopscience.iop.org/article/10.1088/1361-6463/ae11c1
21. Pathways on capacity building for AI supervisory authorities https://unesdoc.unesco.org/ark:/48223/pf0000396637
22. Proceedings of the 2025 Conference on Empirical Methods in ... https://aclanthology.org/volumes/2025.emnlp-main/
23. CRISPR and Artificial Intelligence in Neuroregeneration - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC12524701/
24. BackdoorLLM: A Comprehensive Benchmark for Backdoor Attacks ... https://arxiv.org/html/2408.12798v1
25. Download PDF - Nature https://www.nature.com/articles/d42473-025-00161-3.pdf
26. Perfect Dark (Official Prima Guide) | PDF | Video Games - Scribd https://www.scribd.com/document/339963279/Perfect-Dark-Official-Prima-Guide
27. Walkthrough/FAQ - IGN https://www.ign.com/articles/2003/02/27/perfect-dark-walkthroughfaq-387455

28. Professional Guide / Walkthrough (With Videos) - Steam Community https://steamcommunity.com/sharedfiles/filedetails/?id=2926474173
29. 2021 - Lexia 37-38, "Volti artificiali / Artificial Faces" - Academia.edu https://www.academia.edu/45610149/2021_Lexia_37_38_Volti_artificiali_Artificial_Faces_
30. AI and Big Data: Disruptive Innovation | PDF | Artificial Intelligence https://www.scribd.com/document/807516146/Advances-in-Computational-Intelligence-and-Robotics-Moses-Strydom-Editor-Sheryl-Buckley-Editor-AI-and-Big-Data-s-Potential-for-Disruptive-Inn
31. Steam 创意工坊::All Stuff https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=2948302990
32. (PDF) Designing a game for music : Integrated design approaches ... https://www.researchgate.net/publication/329210926_Designing_a_game_for_music_Integrated_design_approaches_for_Ludic_music_and_Interactivity
33. GBA and DS ROM Hacking Guide - 2016 Edition. - Scribd https://www.scribd.com/document/519603373/romhacking2016-preview1
34. Progress in the Advancement of Porous Biopolymer Scaffold: Tissue ... https://pubs.acs.org/doi/10.1021/acs.iecr.8b05334
35. Sensors, Volume 26, Issue 3 (February-1 2026) – 328 articles - MDPI https://www.mdpi.com/1424-8220/26/3
36. Computer Science - arXiv https://www.arxiv.org/list/cs/new?skip=450&show=1000
37. DataDyne Research: Investigation - Perfect Dark Guide - IGN https://www.ign.com/wikis/perfect-dark/DataDyne_Research:_Investigation
38. A Complete List of All Adversarial Example Papers - Nicholas Carlini https://nicholas.carlini.com/writing/2019/all-adversarial-example-papers.html
39. Published Password Lists: 1 - Ineapple https://ineapple.com/known_pass1
40. Indice dei contenuti per No. 178 in Retro Gamer - Zinio https://www.zinio.com/it/publications/retro-gamer/4664/issues/397818/articles
41. RetroGamer 2018-01-01 (R) 'I' - Scribd https://www.scribd.com/document/368432674/RetroGamer-2018-01-01-R-I
42. The Video Game Explosion | PDF - Scribd https://www.scribd.com/document/257363489/The-Video-Game-Explosion
43. [PDF] Crisis Management in Acute Care Settings - ResearchGate https://www.researchgate.net/profile/Michael-Stpierre/publication/309759721_The_Key_to_Success_Teamwork/links/63e8c23edea6121757a4b902/The-Key-to-Success-Teamwork.pdf
44. [XLS] China https://www.nsfc.gov.cn/Portals/0/fj/fj20170118_03.xls

45. [PDF] IBM Storage DS8000 (10th Generation) Introduction and Planning ... https://www.ibm.com/docs/en/SSY1BJD_10.0/sc2795609.pdf
46. ROG MAXIMUS Z790 FORMULA - Republic of Gamers | ROG Global https://rog.asus.com/motherboards/rog-maximus/rog-maximus-z790-formula/
47. [PDF] 2022 HP Sustainable Impact Report https://h20195.www2.hp.com/v2/getpdf.aspx/c08636600.pdf
48. Building an entity-component-system in Rust - autodeist https://www.autodeist.com/2023/Building-an-entity-component-system-in-Rust/
49. Awesome Rust Overview https://www.trackawesomelist.com/rust-unofficial/awesome-rust/readme/
50. Computer Science - arXiv https://arxiv.org/list/cs/new
51. Seamus Heaney - The Making of the Poet - Springer Nature https://link.springer.com/content/pdf/10.1057/9780230390256.pdf
52. [PDF] Making Games 01-02/17 - Amazon S3 https://s3-eu-west-1.amazonaws.com/editor.production.pressmatrix.com/emags/211203/pdfs/original/fe919d53-2941-4a70-a8bc-a7e87800ab68.pdf
53. New Riders Publishing - Andrew - by Andrew Rollings, Ernest Adam ... https://www.scribd.com/document/410862617/New-Riders-Publishing-Andrew-By-Andrew-Rollings-Ernest-Adam-pdf
54. Euro-Par 2024: Parallel Processing Workshops - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-031-90200-0.pdf
55. Histoire Jeu | PDF | Video Game Platforms - Scribd https://www.scribd.com/document/239610138/Histoire-Jeu
56. working_with_word_embeddings... https://www.kaggle.com/code/ashishrose/working-with-word-embeddings-1
57. 1 - Amazon S3 https://s3.eu-central-1.amazonaws.com/deepset.ai-farm-models/0.4.1/glove-english-uncased-6B/vocab.txt
58. Steam Workshop::ma collection https://steamcommunity.com/workshop/filedetails/?id=1148801229
59. Experts for Op-Gps Trex Enterprises - Linknovate https://www.linknovate.com/search/?query=op-gps%20trex%20enterprises
60. Experts for Nodejs-Mobile-React-Native Plugins Readme - Linknovate https://www.linknovate.com/search/?query=nodejs-mobile-react-native%20plugins%20readme
61. Computer Graphics and Computer Animation:A Retrospective ... https://www.scribd.com/document/453038750/Computer-Graphics-and-Computer-Animation-A-Retrospective-Overview

62. dark shadows | johnaugust.com https://johnaugust.com/?s=dark+shadows
63. [PDF] Persuasive Games - Luis Navarrete https://luisnavarrete.com/USC/pdf/Bogost2.pdf
64. Videogame Development Strategy Guide | PDF - Scribd https://www.scribd.com/document/772044862/Videogame-Developer-s-Strategy-Guide
65. (PDF) Crpg book preview - Academia.edu https://www.academia.edu/33628840/Crpg_book_preview
66. Computer Science - arXiv https://www.arxiv.org/list/cs/new?skip=25&show=1000
67. Bernard Perron (Editor) - Horror Video Games - Essays On ... - Scribd https://www.scribd.com/document/733386564/Bernard-Perron-Editor-Horror-Video-Games-Essays-on-the-Fusion-of-Fear-and-Play-McFarland-Company-2009
68. Entertainment Computing – ICEC 2022 - Springer https://link.springer.com/content/pdf/10.1007/978-3-031-20212-4.pdf
69. Steam 创意工坊::Brock Works https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3576734867
70. Capstone-Assignment - RPubs https://rpubs.com/BushraT/1075426
71. Challenges in Text Analysis - Fully connected NN - Kaggle https://www.kaggle.com/farsanas/challenges-in-text-analysis-fully-connected-nn/notebook
72. Dicionario portugues - Academia.edu https://www.academia.edu/32592435/Dicionario_portugues